



CA-Lab No. 2

Register Operations

2025-03-05

Computer Architecture Class 1403-1

Sharif University of Technology

Autumn 2024

By Dr. Saeed Bagheri Shouraki

I. Objectives

Last session, we developed a « Simple » Calculator. This week, we aim to add some features to our calculator to make it function as an « Engineering » calculator. Specifically, we want to add a few memory cells, design instructions to allow data transfer between the memory cells, and build a combinational multiplier circuit.

II. Pre-Lab Activities

For the following tasks, simulate your design in **Digital** and verify the output results.

You may either use the in-program « testing » feature or manually input a few test vectors.

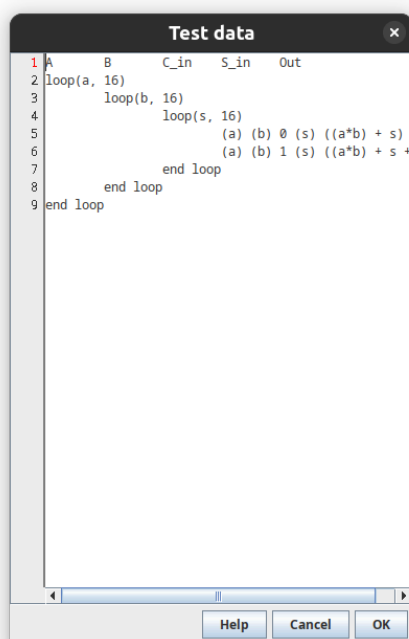


Fig. 1 — Example Test Data

	A	B	C_in	S_in	Out
L5:a=0;b=0...	0	0	0	0	0
L6:a=0;b=0...	0	0	1	0	1
L5:a=0;b=0...	0	0	0	1	1
L6:a=0;b=0...	0	0	1	1	2
L5:a=0;b=0...	0	0	0	2	2
L6:a=0;b=0...	0	0	1	2	3
L5:a=0;b=0...	0	0	0	3	3
L6:a=0;b=0...	0	0	1	3	4
L5:a=0;b=0...	0	0	0	4	4
L6:a=0;b=0...	0	0	1	4	5
L5:a=0;b=0...	0	0	0	5	5
L6:a=0;b=0...	0	0	1	5	6
L5:a=0;b=0...	0	0	0	6	6
L6:a=0;b=0...	0	0	1	6	7
L5:a=0;b=0...	0	0	0	7	7
L6:a=0;b=0...	0	0	1	7	8
L5:a=0;b=0...	0	0	0	8	8
L6:a=0;b=0...	0	0	1	8	9
L5:a=0;b=0...	0	0	0	9	9
L6:a=0;b=0...	0	0	1	9	A
L5:a=0;b=0...	0	0	0	A	A
L6:a=0;b=0...	0	0	1	A	B
L5:a=0;b=0...	0	0	0	B	B
L6:a=0;b=0...	0	0	1	B	C
L5:a=0;b=0...	0	0	0	C	C
L6:a=0;b=0...	0	0	1	C	D
L5:a=0;b=0...	0	0	0	D	D
L6:a=0;b=0...	0	0	1	D	E
L5:a=0;b=0...	0	0	0	E	E

Fig. 2 — Test Results

II.1. Combinational Multiplier Design

Design a 4-bit multiplier circuit from scratch that takes two numbers (A , B) and computes their product (R). You can **only use Adders, MUXes, and Barrel Shifters** along with the necessary I/O components (buttons, splitters, etc.).

As a hint, note that you can multiply two 4-bit numbers using only addition and binary left shifting:

$$\begin{aligned}
A &= 0b1110 = 14 \\
B &= 0b1010 = 10 \\
\Rightarrow A &= 1 \times 2^3 + 1 \times 2^2 + 1 \times 2^1 + 0 \times 2^0 \\
\Rightarrow B &= 1 \times 2^3 + 0 \times 2^2 + 1 \times 2^1 + 0 \times 2^0 \\
\Rightarrow A \times B &= 2^3 \times B + 2^2 \times B + 2^1 \times B \\
\Rightarrow A \times B &= \text{ShiftLeft}(B, 3) + \text{ShiftLeft}(B, 2) + \text{ShiftLeft}(B, 1) \\
\Rightarrow A \times B &= 0101_0000 + 0010_1000 + 0001_0100 \\
\Rightarrow A \times B &= 140
\end{aligned}$$

Note that for 4-bit input signals, you must provide an 8-bit output signal. All inputs are assumed to be unsigned for now.

In your report:

- Include a screenshot of your implementation in Digital.
- Include a screenshot of your implementation's output for $A = 3$ and $B = 11$.

II.2. Registers

Registers will act as the memory cells of your calculator. You can save values inside the registers or retrieve previously saved values from one of the registers to perform arithmetic functions.

Draw a circuit consisting of **8 8-bit Registers**, numbered R_0 to R_7 . At each « operational cycle » (a single clock cycle or multiple clock cycles if an operation requires multiple steps), the circuit must be able to accomplish one of the following tasks:

- Take an 8-bit **input** (from buttons) and store it in register R_0 .
- Take the contents of R_1 and R_2 , **add** them together (discarding the carry bit), and store the result in R_3 .
- Take the contents of R_1 and R_2 , **subtract** R_2 from R_1 (discarding the carry bit), and store the result in R_3 .
- **Copy** the contents of R_0 to R_1 .
- Copy the contents of R_3 to R_2 .

You can use a counter and an EEPROM to apply the control signals automatically.

Important Note!

You must build a single circuit capable of accomplishing all of the above by just manipulating the control signals. For example, you must be able to first put number 7 to R_0 , copy it to R_1 , add it to R_2 and store the result in R_3 . You cannot change the wiring of the circuit or add components between the operations.

In your report:

- Explain the inner workings of your design.
- Include a screenshot of your implementation in Digital.
- Walk through the process of accomplishing one of the above tasks by expressing the required control signals on a clock-by-clock basis.

II.3. Register Transfer Layer (RTL)

In this task, we will implement general transfers (or movement) of data between the registers.

Modify your circuit so that, at each « operational cycle, » your circuit can copy the contents of any register R_i to R_j . Note that i can be equal to j , in which case the contents of the register must remain unchanged except possibly when $i = j = 0$.

In your report:

- Explain the inner workings of your design.
- Include a screenshot of your implementation in Digital.
- Walk through the process of completing a single transfer by expressing the required control signals on a clock-by-clock basis.

Hint

You should implement a « databus » mechanism consisting of a single wire connected (perhaps via a multiplexer) to every register's **D** signal. At each moment in time, at most one register's output should drive the databus through a multiplexer (only one register can transmit data at a time). You can transfer the contents of R_i to R_j by allowing R_i to drive the databus (taking control of it), then setting the **en** pin of R_j to **1** (while setting the **en** pin of every other register to **0**) and waiting for one clock cycle.